

Lancelot Dallain RL TP 2

Partie 1 : Jeu de Pacman simplifié

1.1 Pour commencer

Conformément à l'énoncé, l'état du système est défini à partir de la position du Pacman et de l'état des nourritures présentes dans l'environnement. On note F_i la variable binaire associée à la nourriture numéro i , avec $i \in [0, n - 1]$, telle que $F_i = 0$ lorsque la nourriture est intacte et $F_i = 1$ lorsqu'elle a été consommée. La position du Pacman est représentée par la variable Pos , correspondant à l'indice de la case occupée sur le damier, avec la convention $Pos = 0$ pour la case située en bas à gauche.

Le calcul de l'état global est obtenu à l'aide de la formule suivante :

$$State = Pos + N_{Pos} \times \sum_{i=0}^{n-1} F_i \times 2^i$$

où N_{Pos} désigne le nombre total de positions possibles sur le damier et n le nombre de nourritures.

Ce codage permet de représenter l'état complet du jeu, car il distingue à la fois toutes les positions possibles du Pacman et toutes les combinaisons possibles des états des nourritures, mangées ou non. Chaque configuration physique du jeu est ainsi associée à un état unique, ce qui garantit une représentation exhaustive de l'espace d'état et satisfait la propriété de Markov requise pour l'apprentissage par renforcement.

En utilisant cette formule et en se basant sur l'environnement décrit par le fichier *board0.txt*, qui correspond à un damier minimaliste de taille 2×2 comportant une unique nourriture, on obtient :

$$State = Pos + (2 \times 2) \times F_0 \times 2^0 = Pos + 4 \times F_0$$

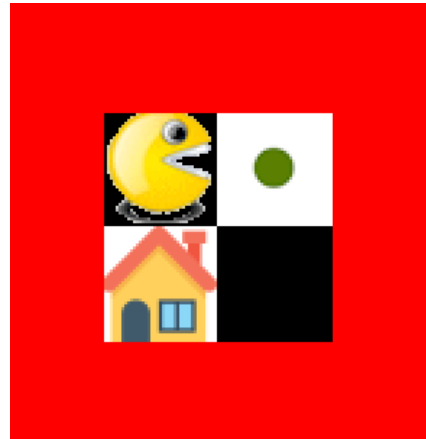
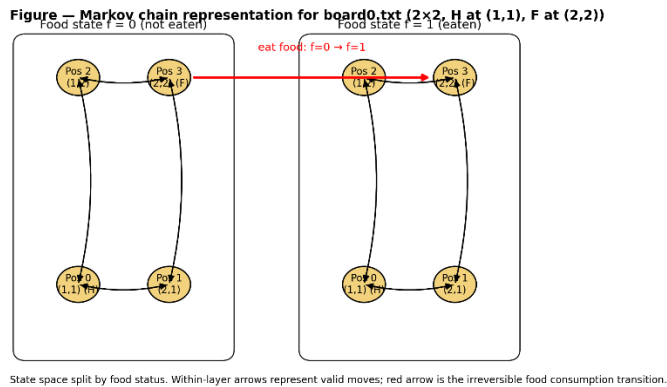
Le nombre théorique total d'états est alors donné par le produit du nombre de positions possibles et du nombre de configurations des nourritures, soit :

$$N_{State} = N_{Pos} \times 2^n = 4 \times 2 = 8$$

Cependant, l'exploration de l'environnement en pratique montre que tous ces états théoriques ne sont pas nécessairement atteignables. En effet, lorsque le Pacman se déplace sur la case contenant la nourriture, celle-ci est automatiquement consommée, ce qui empêche certaines configurations de persister dans le temps. De plus, le jeu se termine dès que la nourriture a été consommée et que le Pacman rejoint sa maison. Ces configurations correspondent à des états terminaux, également

appelés états absorbants, qui interrompent immédiatement l'épisode et limitent l'exploration de l'espace d'état.

Dans le cas de l'environnement *board0.txt*, cette dynamique implique qu'un des états théoriques n'est jamais observé au cours de l'exécution du jeu, conduisant ainsi à un nombre effectif d'états inférieur au nombre théorique. La dynamique globale du jeu peut alors être modélisée par une chaîne de Markov finie, mettant en évidence les transitions possibles entre les états accessibles ainsi que la présence d'états absorbants correspondant à la fin de la partie.



1.2 Apprentissage de la table action-value function (Q)

La fonction Qlearning implémente l'algorithme de **Q-learning par Temporal Difference**, permettant d'estimer la fonction action-valeur Q sans connaître la matrice de transition de l'environnement.

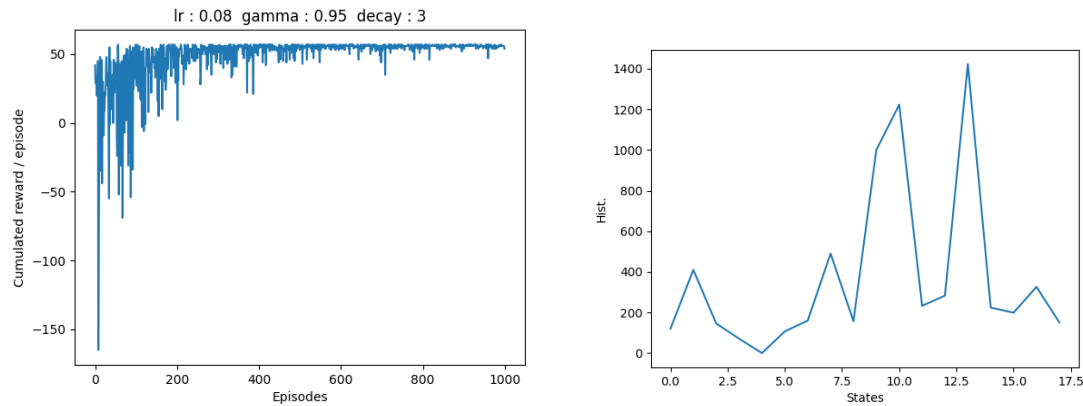
La politique ϵ -greedy assure un compromis entre exploration et exploitation, tandis que la mise à jour TD corrige progressivement l'estimation de Q à partir des récompenses observées.

On utilise maintenant le fichier board1.txt, qui décrit en environnement 3x3 avec 1 nourriture et 1 poison, pour lequel on lance un apprentissage. On obtient une Q table qui décrit les rewards associées à chaque état et chaque actions (cf. figure 5). La dimension de la table est donc ($N_{brState}$, $N_{brActions}$)

Après apprentissage, les statistiques calculées sur 100 parties montrent une récompense cumulée moyenne de 56.46, un taux de victoire de 100 %, un nombre moyen de déplacements égal à 4.54 et aucun passage sur une case poison. Ces résultats indiquent que le Pacman a appris une stratégie efficace, lui permettant de manger la nourriture puis de rejoindre la maison en un nombre réduit de déplacements tout en évitant systématiquement le poison.

L'évolution de la récompense cumulée au fil des épisodes présente une forte variabilité au début de l'apprentissage, ce qui s'explique par la phase d'exploration durant laquelle les actions sont majoritairement aléatoires. Lorsque le nombre d'épisodes augmente, la récompense cumulée se stabilise autour d'une valeur élevée, traduisant la convergence progressive de la politique vers une stratégie optimale. Quelques valeurs plus faibles subsistent ponctuellement, ce qui est dû à l'exploration résiduelle imposée par la politique ϵ -greedy.

L'histogramme des états visités montre que certains états sont beaucoup plus fréquentés que d'autres. Cette non-uniformité s'explique par le fait que la politique apprise privilégie des trajectoires courtes reliant les positions initiales à la nourriture puis à la maison, tout en évitant la case poison, ce qui rend certains états peu visités voire quasiment inaccessibles en phase d'exploitation.



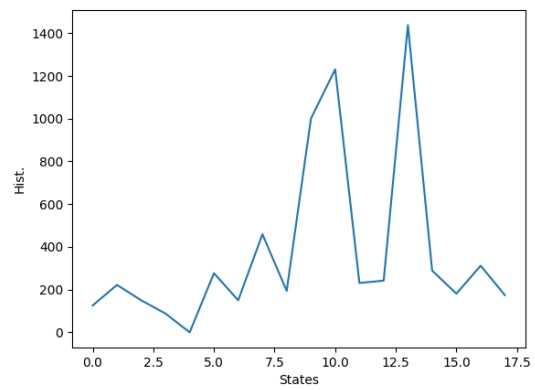
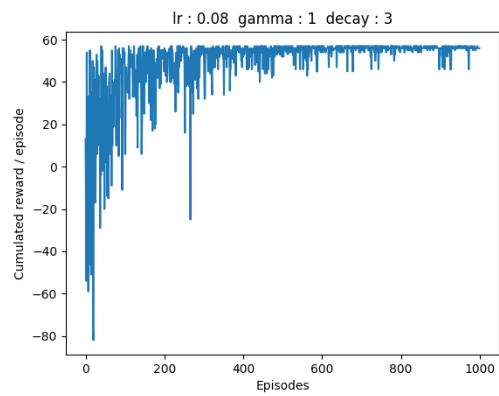
B-3

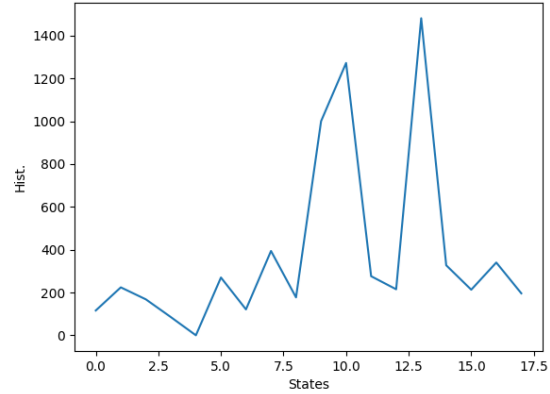
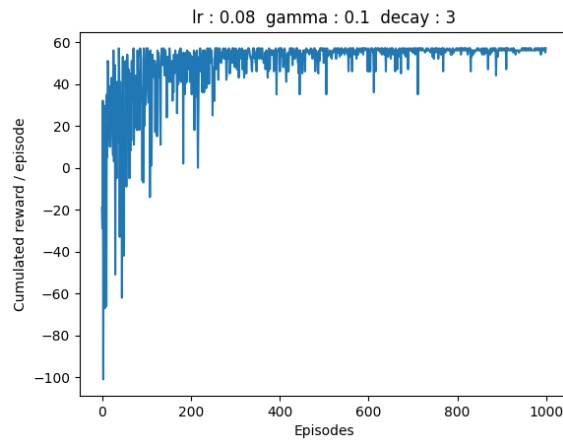
Lorsque l'on prend $\gamma = 1$, les récompenses futures ne sont pas actualisées. L'agent cherche donc à maximiser la somme totale des récompenses jusqu'à la fin de l'épisode. La courbe d'apprentissage montre une forte variabilité au début, due à l'exploration, puis une stabilisation autour d'une récompense cumulée élevée. Les statistiques sur 100 parties indiquent une récompense moyenne d'environ 56.4, un taux de victoire de 100 %, un nombre moyen de déplacements proche de 4.6 et aucun passage sur une case poison. La matrice Q contient des valeurs élevées, reflétant des retours cumulés importants, en particulier pour les actions appartenant aux trajectoires optimales.

Les valeurs de la matrice Q ne correspondent pas exactement à des valeurs théoriques uniques, car elles sont obtenues par apprentissage à partir d'épisodes échantillonnés. Elles dépendent de la politique d'exploration, du nombre fini d'épisodes et du fait que certains états sont peu visités, ce qui explique la présence de lignes peu ou pas mises à jour.

Lorsque l'on prend $\gamma = 0.1$, les récompenses futures sont fortement décotées et l'agent privilégie davantage le gain immédiat. La courbe d'apprentissage reste globalement similaire et les statistiques finales sont très proches de celles obtenues avec $\gamma = 1$, avec un taux de victoire de 100 % et une absence totale de poison. En revanche, la matrice Q est très différente : la plupart des valeurs sont proches des récompenses immédiates, souvent négatives à cause du coût de déplacement, et seules les actions menant directement à la nourriture ou à la victoire conservent des valeurs élevées.

Cette différence de Q-table est normale et provient directement du rôle du facteur d'actualisation γ dans la mise à jour de Q. Dans ce cas précis, la différence a peu d'impact sur la politique finale car le problème est simple, les épisodes sont courts et la stratégie optimale est peu ambiguë. Pour des environnements plus complexes ou nécessitant des sacrifices à court terme pour un gain futur, le choix de γ aurait en revanche une influence beaucoup plus marquée sur la politique apprise.



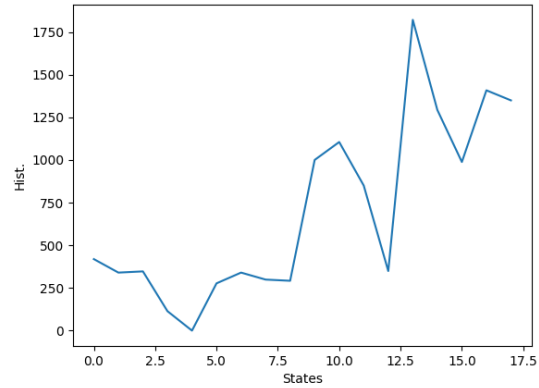
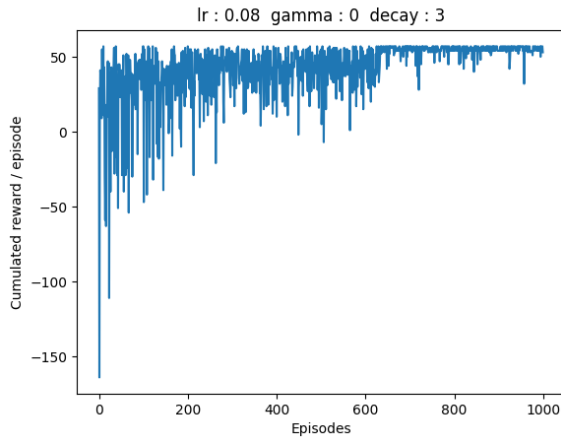


B-4

Lorsque l'on prend $\gamma = 0$, les récompenses futures ne sont plus prises en compte et l'agent n'optimise que la récompense immédiate associée à chaque action. La courbe d'apprentissage montre une variabilité importante sur l'ensemble des épisodes et une stabilisation plus lente et moins nette que pour $\gamma = 1$ ou $\gamma = 0.1$. Les statistiques sur 100 parties indiquent une récompense cumulée moyenne de 24.88, un taux de victoire de 70 % et un nombre moyen de déplacements élevé, d'environ 18.1, bien supérieur aux cas précédents.

La matrice Q obtenue reflète ce comportement myope. Les valeurs sont principalement proches des récompenses immédiates, avec des pénalités liées aux déplacements et des valeurs élevées uniquement pour les actions conduisant directement à une récompense forte, comme la nourriture ou la victoire lorsqu'elles sont atteignables en un seul pas. L'agent ne valorise plus le fait de se rapprocher progressivement d'un objectif lointain, ce qui explique les trajectoires plus longues, moins efficaces et le taux de victoire réduit.

Ces résultats montrent que le facteur d'actualisation γ joue un rôle essentiel dans l'apprentissage d'une stratégie cohérente sur l'ensemble de l'épisode. Lorsque γ est nul, l'agent n'est plus capable d'anticiper les bénéfices futurs de ses actions, ce qui dégrade fortement la qualité de la politique apprise, même dans un environnement simple comme celui-ci.

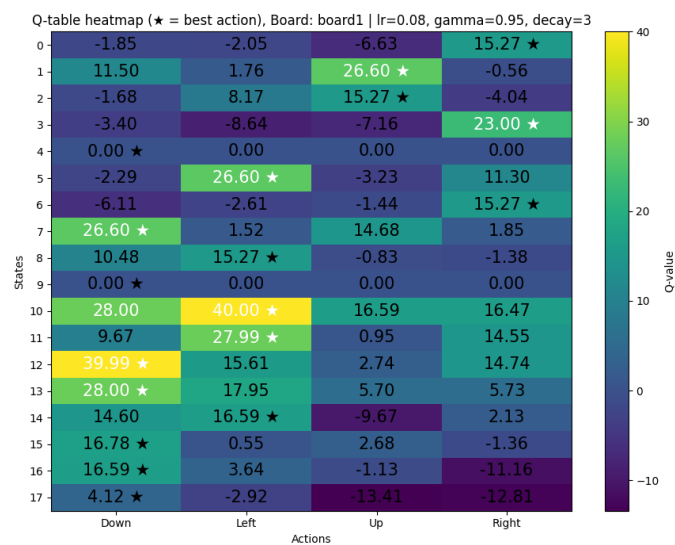


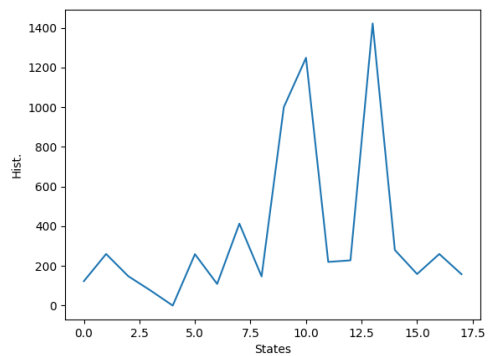
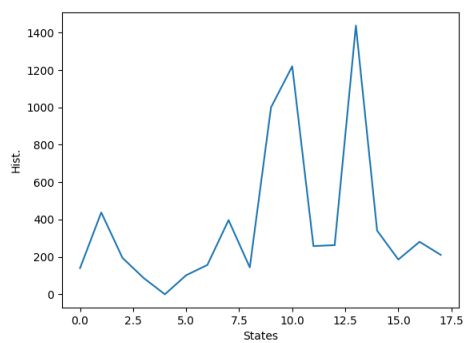
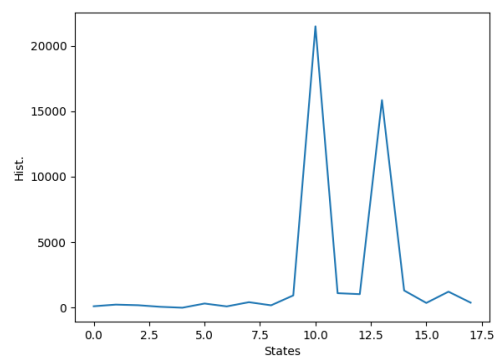
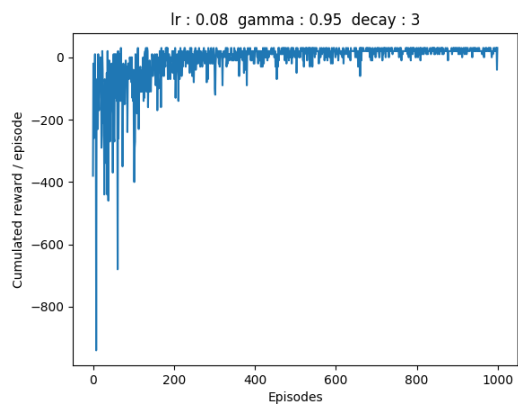
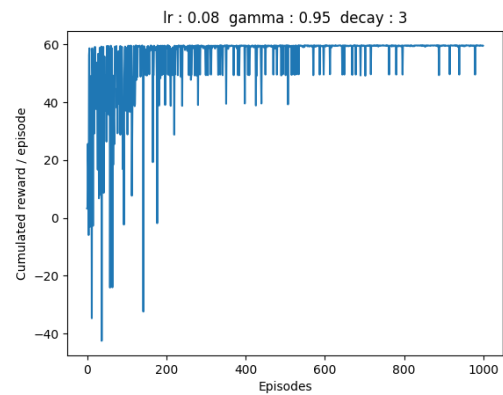
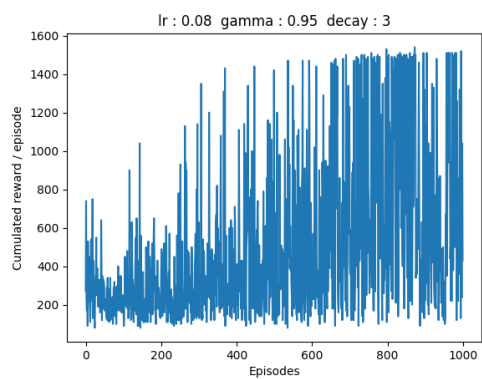
B-5

Pour étudier l'influence des rewards, on fixe les hyperparamètres d'apprentissage (notamment $\gamma = 0,95$) et on ne fait varier que le **STEP_REWARD** sur board1. Avec **STEP_REWARD** = **-0.1**, on obtient une récompense cumulée moyenne de **59.644**, un taux de victoire de **100 %**, **4.56** déplacements en moyenne et **0** case poison. Le coût du mouvement étant faible, la politique optimale (manger puis rentrer) reste inchangée, mais le score final est mécaniquement plus élevé car chaque déplacement "coûte" moins. Avec **STEP_REWARD** = **-10**, la victoire reste à **100 %** et le nombre de déplacements moyen reste similaire (**4.6**), mais la récompense cumulée moyenne chute à **24.0** : l'agent réalise toujours essentiellement le même chemin minimal, mais chaque pas est fortement pénalisé, ce qui dégrade fortement le score malgré une stratégie correcte. À l'inverse, avec **STEP_REWARD** = **+10**, on observe un comportement pathologique : la récompense cumulée moyenne devient **500.0**, le taux de victoire tombe à **0 %**, le nombre de déplacements moyen atteint **50.0**, et le poison reste à **0**. L'agent est alors incité à maximiser le nombre de pas plutôt qu'à terminer l'épisode, car se déplacer rapporte plus que finir, ce qui illustre qu'un reward mal défini peut conduire à une politique "optimale" au sens du reward mais contraire à l'objectif du jeu.

B-6

Pour board2, la taille vaut 5, donc le damier est de 5×5 et il y a $N_{Pos} = 25$ positions possibles. On observe deux nourritures (F) en (4,5) et (2,5), donc $n = 2$ variables binaires F_0, F_1 . Le poison n'entre pas dans la définition de l'état. Avec le codage $State = Pos + N_{Pos} \sum_{i=0}^{n-1} F_i 2^i$, le nombre d'états théoriques est donc $25 \times 2^2 = 100$.



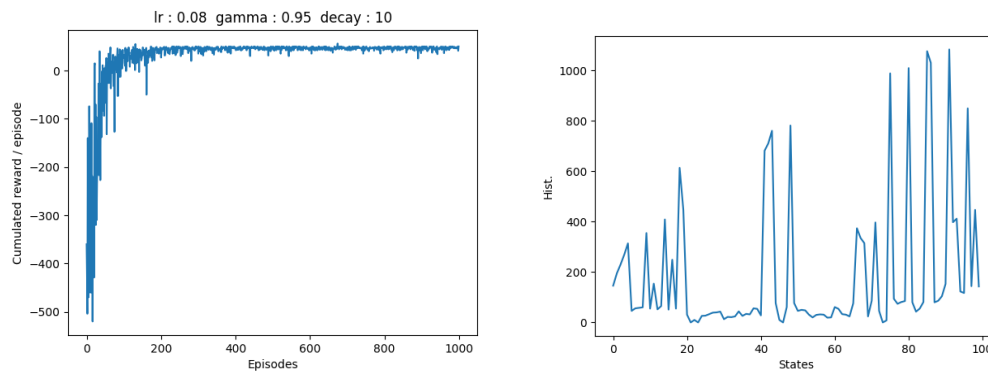


B-7

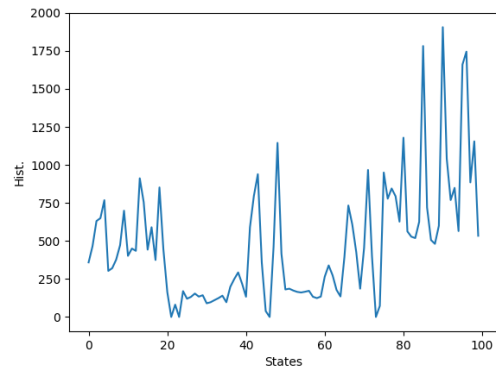
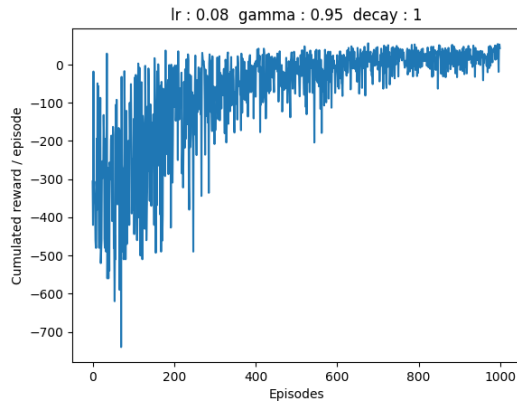
Le paramètre `exploration_decreasing_decay` contrôle la vitesse à laquelle l'agent passe du mode exploration au mode exploitation. Pour une valeur faible ($\text{decay} = 1$), l'exploration reste importante pendant un grand nombre d'épisodes, ce qui se traduit par une courbe d'apprentissage très bruitée et une convergence plus lente, même si la politique finale est correcte. À l'inverse, pour une valeur élevée ($\text{decay} = 10$ ou 100), l'agent exploite rapidement sa politique, ce qui entraîne une stabilisation plus rapide de la récompense cumulée, au prix d'une exploration initiale plus limitée.

Dans ce problème, on observe que pour une large gamme de valeurs de `exploration_decreasing_decay`, les performances finales restent proches, avec un taux de victoire de 100 % et des trajectoires comparables. Cela s'explique par la structure relativement simple de l'environnement et par un signal de récompense très informatif, qui permet à l'agent d'identifier rapidement les bonnes actions même avec peu d'exploration. Ainsi, tant que l'exploration n'est pas supprimée trop brutalement, la Q-table converge vers une politique satisfaisante malgré des vitesses de décroissance très différentes.

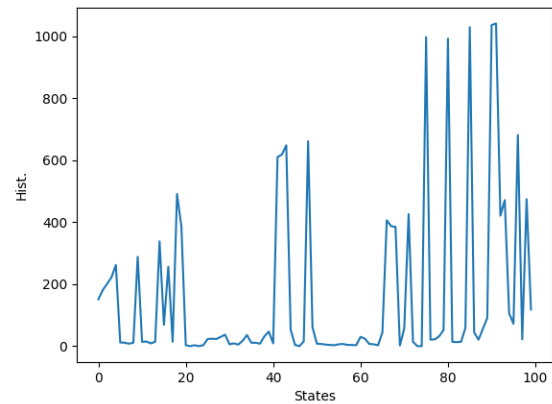
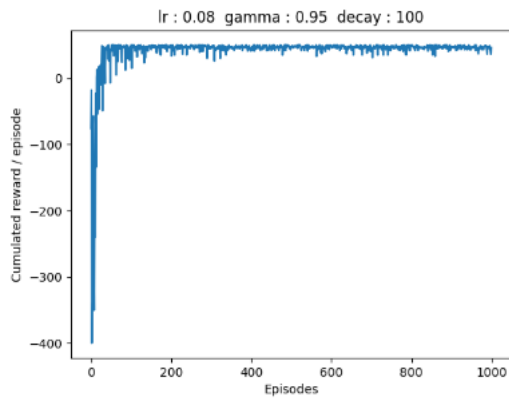
Calcul des stat sur 100 parties
Récompense cumulée moyenne : 46.79 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 14.21
Nombre moyen de cases poison par partie 1.0



Calcul des stat sur 100 parties
Récompense cumulée moyenne : 49.58 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 14.32
Nombre moyen de cases poison par partie 0.71



Calcul des stat sur 100 parties
Récompense cumulée moyenne : 47.22 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 13.78
Nombre moyen de cases poison par partie 1.0



Avec 0.1
Calcul des stat sur 100 parties
Récompense cumulée moyenne : 48.59 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 15.01
Nombre moyen de cases poison par partie 0.74

B-8 Pour éviter que le Pacman ne s’empoisonne, il est nécessaire que la pénalité associée au poison soit suffisamment forte par rapport aux autres rewards, en particulier au coût de déplacement et aux récompenses liées à la nourriture ou à la victoire. Si la valeur absolue du POISON_REWARD est trop faible, l’agent peut considérer le passage par une case poison comme acceptable afin de réduire le nombre de déplacements. À l’inverse, lorsque le poison est fortement pénalisé, son impact négatif domine la fonction de récompense et l’agent apprend à éviter systématiquement ces cases, même si cela implique des trajectoires plus longues. Plus généralement, il faut veiller à ce que le coût du poison soit supérieur au gain potentiel obtenu en raccourcissant le chemin, afin que l’empoisonnement ne soit jamais une option optimale du point de vue de la récompense cumulée.

B-9

Calcul des stat sur 100 parties
Récompense cumulée moyenne : 51.48 Pourcentage victoire : 99.0 %
Nombre de déplacements moyens 38.62
Nombre moyen de cases poison par partie 0.0

PARTIE 2 :

A/ Analyse du fichier DQNPtorch.py

Le fichier DQNPtorch.py met en œuvre un algorithme de Deep Q-Learning dans lequel la fonction action-valeur est approximée par un réseau de neurones. L’agent interagit avec l’environnement en suivant une politique ϵ -greedy : l’action est choisie aléatoirement avec une probabilité ϵ , ou bien par maximisation des valeurs de Q prédites par le réseau dans le cas contraire. Le taux d’exploration décroît progressivement au cours de l’apprentissage.

Chaque interaction génère une transition composée de l’état courant, de l’action effectuée, de la récompense reçue, de l’état suivant et d’un indicateur de terminaison. Ces transitions sont stockées dans une mémoire de type replay buffer, puis rééchantillonnées aléatoirement sous forme de mini-batches afin d’entraîner le réseau de neurones. Cette stratégie permet de limiter les corrélations temporelles entre observations successives.

L’apprentissage repose sur la minimisation de l’erreur entre la valeur de Q associée à l’action réalisée et une cible temporelle construite à partir de la récompense immédiate et de la valeur maximale estimée de l’état suivant. Cette dernière est calculée à l’aide d’un réseau cible, mis à jour périodiquement à partir du réseau principal, ce qui contribue à stabiliser l’apprentissage. Les paramètres du réseau sont ajustés par descente de gradient à chaque itération.

L’entraînement est structuré par épisodes, chacun étant limité à un nombre maximal d’itérations. Les récompenses cumulées par épisode permettent d’évaluer la progression de l’apprentissage et la qualité de la politique apprise.

B/

Le replay buffer sert à stocker les transitions $(s, a, r, s', done)$ générées au fil des épisodes et à entraîner le réseau sur des mini-batches tirés aléatoirement dans cette mémoire plutôt que sur les transitions les plus récentes uniquement. Dans ton code, chaque pas ajoute une expérience via `memory.add(...)`, puis `learn()` récupère un batch avec `memory.sample()`. Ce rééchantillonnage aléatoire casse la forte corrélation entre états successifs d'un même épisode et permet de réutiliser des expériences plus anciennes, ce qui rend l'apprentissage par réseau de neurones plus stable que si l'on apprenait uniquement "en ligne" sur une séquence temporelle. On voit aussi que la terminalité est intégrée sous forme de masque : `done = 1 - done`, ce qui permet ensuite d'annuler le terme de bootstrap dans la cible lorsque l'épisode est terminé.

Le réseau target a pour rôle de stabiliser la construction des cibles temporelles utilisées pour entraîner le réseau principal. Dans ton implémentation, les valeurs $Q(s, a)$ à ajuster sont produites par `self.network(states)`, tandis que la partie "valeur future" de la cible est calculée avec `self.target_network(states_)`. Le réseau target est initialisé comme une copie du réseau principal puis mis à jour de manière périodique seulement, toutes les `TARGET_UPDATE_FREQ` itérations, par copie des poids. Cette séparation évite que la cible utilisée pour entraîner le réseau ne change brutalement à chaque mise à jour des poids du modèle, ce qui limite les oscillations et les instabilités de convergence.

Pour la board1, l'état vectoriel a une dimension $2 + \text{nombre de nourritures}$. Comme il y a une seule nourriture, l'entrée du réseau est de dimension 3 et la sortie est de dimension 4 (les 4 actions). Avec l'architecture MLP à deux couches cachées de 16 neurones, le nombre de paramètres (poids et biais) vaut $(3 \times 16 + 16) + (16 \times 16 + 16) + (16 \times 4 + 4) = 404$. À comparer, dans la partie 1 avec codage scalaire, le nombre d'états pour un damier 3×3 avec une nourriture est $9 \times 2 = 18$, et la Q-table contient donc $18 \times 4 = 72$ valeurs. Cette comparaison montre que l'approche tabulaire stocke explicitement une valeur par couple (état, action), alors que l'approche DQN remplace cette table par une approximation paramétrique, dont la complexité dépend de l'architecture du réseau plutôt que du nombre d'états.

Question (c)

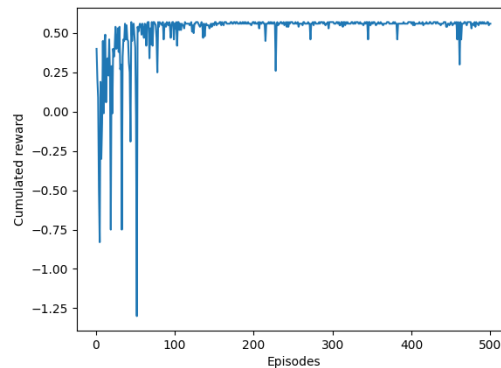
Avec $\gamma = 0.95$ et une architecture **16×16** sur **board1**, l'apprentissage est correct et converge de façon stable. La courbe de récompense cumulée présente une phase initiale très bruitée, avec quelques épisodes fortement négatifs, puis une stabilisation rapide autour de **0.5–0.6** sur la grande majorité des épisodes, ce qui traduit l'acquisition d'une politique performante malgré une exploration résiduelle.

Les sorties de fin d'entraînement confirment cette convergence : à l'épisode 500, on obtient **Average Reward = 0.5, Best Reward = 0.6, Last Reward = 0.6**, avec un taux d'exploration minimal atteint ($\epsilon = 0.05$). La perte finale est faible (**Loss $\approx 2.66 \times 10^{-4}$**), ce qui indique que l'optimisation du réseau s'est stabilisée.

L'évaluation statistique valide également le comportement appris : sur 100 parties, le **score moyen est ≈ 0.5646** , le **taux de victoire est de 100 %**, le **nombre moyen de déplacements est de 3.54**, et le **nombre moyen de cases poison traversées est nul (0.0)**. L'agent termine donc systématiquement le jeu, avec un chemin court, et sans passer sur le poison.

Si l'on change la position d'un **poison**, il est nécessaire de **ré-apprendre** (ou au minimum d'affiner) le réseau, car la représentation d'état utilisée $[X, Y, F_i]$ ne décrit pas explicitement la géométrie du plateau ni la localisation des dangers : déplacer un poison modifie la structure des récompenses associée aux trajectoires possibles, donc la fonction $Q(s, a)$ apprise n'est plus adaptée. La même conclusion s'applique à un changement de position de la **maison**, qui modifie la condition de terminaison et les trajectoires optimales sans être encodé dans l'état.

GAMMA 0.95
Neurones 16 16
Done Episode 500 Average Reward 0.5 Best Reward 0.6 Last Reward 0.6 Epsilon 0.05
Bravo Loss = 0.0002657034856383689
Episode 500/500 Cum. Reward 0.6 Epsilon 0.05
Score moyen : 0.5646000000000001 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 3.54
Nombre moyen de cases poison par partie 0.0
board used: board1



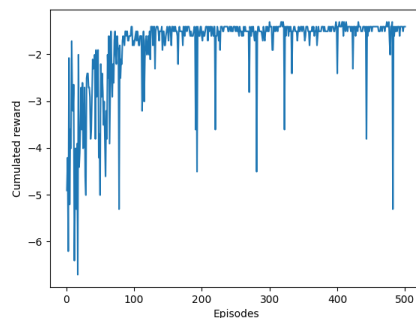
D

Sur **board2** avec $\gamma = 0.95$, l'apprentissage n'est pas correct avec un réseau de petite taille (3 neurones par couche). Les sorties indiquent que l'agent n'arrive jamais à terminer la partie : le mode stat donne un **taux de victoire de 0 %**, un **nombre moyen de déplacements de 50** (ce qui correspond au plafond imposé par la boucle de test), et un **score moyen négatif** (≈ -0.493). Le log d'entraînement confirme ce blocage avec le message "**Max step atteint**" à la fin, et une récompense cumulée finale **Cum. Reward** = **-1.4** à l'épisode 500. Autrement dit, la politique apprise ne mène pas à l'état terminal et le Pacman "survit" sans gagner, ce qui est cohérent avec une capacité d'approximation insuffisante du réseau pour cette configuration plus complexe.

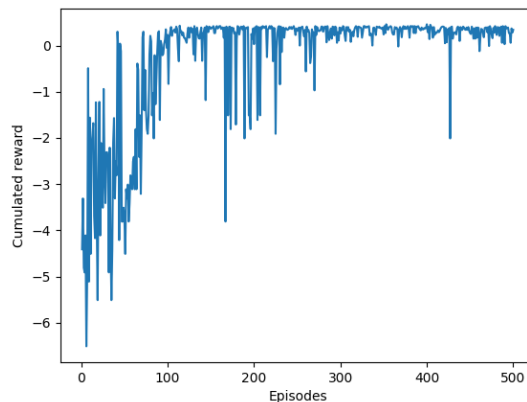
En augmentant la capacité du modèle avec un réseau **8×8**, l'apprentissage devient en revanche correct. À l'épisode 500 on obtient **Average Reward** = **0.2**, **Best Reward** = **0.5**, **Last Reward** = **0.3**, et l'évaluation sur 100 parties affiche un **taux de victoire de 100 %**, un **score moyen ≈ 0.3913** , et **10.87 déplacements moyens**, sans passage sur le poison (**0.0**). Cela montre que l'échec précédent provenait principalement d'un réseau trop petit pour représenter une fonction $Q(s, a)$ suffisamment discriminante sur board2.

L'observation en mode *play* est cohérente avec ces résultats : avec le petit réseau, la raison principale de l'échec est que le Pacman ne parvient pas à atteindre une séquence d'actions menant à la terminaison et finit par **boucler ou errer** jusqu'à atteindre la limite de pas, ce qui empêche toute victoire. Avec le réseau 8×8, la politique devient suffisamment structurée pour collecter les objectifs et terminer la partie dans un nombre de pas modéré.

GAMMA 0.95
Neurones 3, 3
Max step atteint Loss = 0.014381067252088542
Episode 500/500 Cum. Reward -1.4 Epsilon 0.05
Saved in models/board2.weights
Score moyen : -0.4930000000000001 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board2

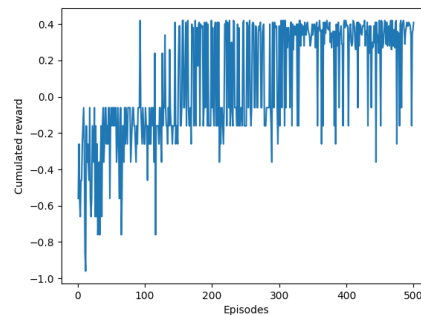


GAMMA 0.95
Neurones 8, 8
Done Episode 500 Average Reward 0.2 Best Reward 0.5 Last Reward 0.3 Epsilon 0.10
Bravo Loss = 0.005065416356956121
Episode 500/500 Cum. Reward 0.3 Epsilon 0.10
Saved in models/board2.weights
Score moyen : 0.3913 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 10.87
Nombre moyen de cases poison par partie 0.0
board used: board2

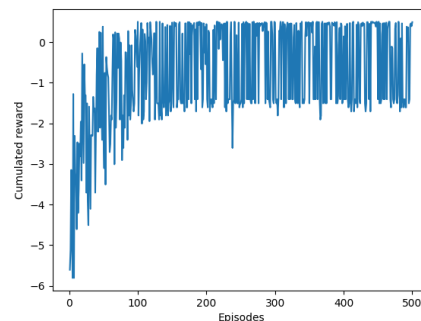


E/

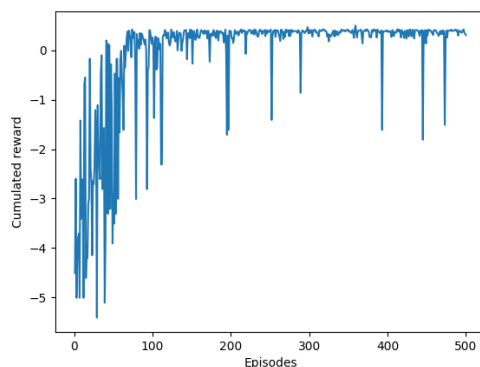
GAMMA 0.95
Neurones 16,16
MAX STEP 15
BUFFER 4096
Done Episode 500 Average Reward 0.2 Best Reward 0.4 Last Reward 0.4 Epsilon 0.05
Bravo Loss = 0.0004688523022196023
Episode 500/500 Cum. Reward 0.4 Epsilon 0.05
Score moyen : 0.3697 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 10.93
Nombre moyen de cases poison par partie 0.0
board used: board2



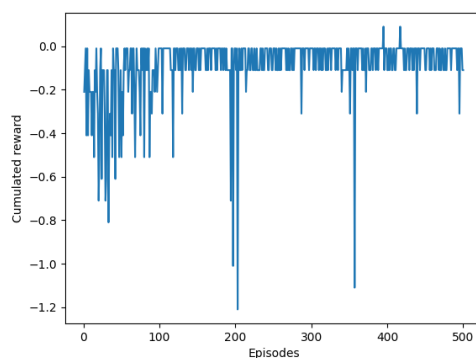
GAMMA 0.5
Neurones 16,16
MAX STEP 150
BUFFER 4096
Done Episode 500 Average Reward 0.1 Best Reward 0.5 Last Reward 0.5 Epsilon 0.05
Bravo Loss = 5.274075454053673e-05
Episode 500/500 Cum. Reward 0.5 Epsilon 0.05
Score moyen : 0.08859999999999993 Pourcentage victoire : 57.0 %
Nombre de déplacements moyens 28.54
Nombre moyen de cases poison par partie 0.0
board used: board2



GAMMA 0.95
Neurones 16,16
MAX STEP 150
BUFFER 409
Done Episode 500 Average Reward 0.2 Best Reward 0.5 Last Reward 0.3 Epsilon 0.05
Bravo Loss = 0.00048639935312166926
Episode 500/500 Cum. Reward 0.3 Epsilon 0.05
Score moyen : 0.3904999999999996 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 10.95
Nombre moyen de cases poison par partie 0.0
board used: board2



GAMMA 0.5
Neurones 16,16
MAX STEP 10
BUFFER 409
Max step atteint Loss = 0.0005968695750198094
Episode 500/500 Cum. Reward -0.1 Epsilon 0.05
Score moyen : -0.42200000000000015 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board2



GAMMA 0.95
Neurones 16,16
MAX STEP 10
BUFFER 409
Max step atteint Loss = 0.0008384605989704141
Episode 500/500 Cum. Reward -0.0 Epsilon 0.05
Score moyen : -0.40000000000000002 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board2

